```
      AttributeDefinitions:
      - AttributeName: id                                    ⑥
        AttributeType: S
      KeySchema:
      - AttributeName: id
        KeyType: HASH
      ProvisionedThroughput:
        ReadCapacityUnits: 1
        WriteCapacityUnits: 1
      TableName: 'imagery-image'
  SQSDLQueue:                                                ⑦
    Type: 'AWS::SQS::Queue'
    Properties:
      QueueName: 'imagery-dlq'
  SQSQueue:                                                  ⑧
    Type: 'AWS::SQS::Queue'
    Properties:
      QueueName: imagery
      RedrivePolicy:
        deadLetterTargetArn: !Sub '${SQSDLQueue.Arn}'
        maxReceiveCount: 10                                  ⑨
# [...]
Outputs:
  EndpointURL:                                               ⑩
    Value: !Sub 'http://${LoadBalancer.DNSName}'
    Description: Load Balancer URL
```

① The map contains key-value pairs mapping regions to AMIs built by us including the Imagery server and worker.

② The CloudFormation template contains a typical public VPC configuration.

③ A S3 bucket for uploaded and processed images, with web hosting enabled

④ The bucket name contains the account ID to make the name unique.

⑤ DynamoDB table containing the image processes

⑥ The id attribute is used as the partition key.

⑦ The SQS queue that receives messages that can’t be processed

⑧ The SQS queue to trigger image processing

⑨ If a message is received more than 10 times, it’s moved to the dead-letter queue.

⑩ Visit the output with your browser to use Imagery.

The concept of a *dead-letter queue (DLQ)* needs a short introduction here as well. If a single SQS message can’t be processed, the message becomes visible again on the queue after reaching its